

AI Manual Troubleshooting Prototype

Sample built from yacht generator and air-conditioning PDF manuals. The system trains a text model and provides a dashboard where a user can type a technical issue and receive detailed manual-based troubleshooting guidance.

Prepared by	Date	Project status
Usama	22 May 2026	Working local prototype

Purpose: This report summarizes the sample I created for the requested AI system. It explains the data used, model training, testing result, and dashboard behavior in a simple flow.

1. Data Used

The prototype uses four PDF manuals placed in the project folder. These manuals cover yacht/marine equipment including a Mase VS 22 LOW generator and Frigomar air-conditioning/chiller/fan-coil systems.

PDF manual	Text chunks created
43992 - VS22LOW_eng rev.2 - Use, Maintenance & Installation manual 1.pdf	105
44149 - VS 22 LOW IT-EN 1.pdf	445
FANCOIL AH SERIES MANUAL REV20210428 1.pdf	119
FRIGOMAR ENG - CU50VFD-CU70VFD 20220111 3.pdf	61

The PDF text was extracted page by page, cleaned, and split into smaller chunks. These chunks became the base training data and also the searchable manual knowledge index.

2. Model Training

I used a practical text classification model from scikit-learn. The model first converts manual text into numerical text features, then learns how different service topics are written in the manuals.

Part	Used	Reason
Text features	TfidfVectorizer	Turns manual text into useful word/phrase features.
Classifier	MultinomialNB	Good baseline for text classification, fast, explainable, and reliable for small document datasets.
Search layer	BM25-style retrieval	Finds the most relevant manual passages for the user problem.
Answer layer	Python resolver API	Builds a detailed answer with summary, possible causes, diagnostic steps, safety notes, and PDF/page evidence.

The classifier detects the issue area, for example cooling water, fuel system, electrical wiring, alarms, spare parts, maintenance, or troubleshooting. The retrieval layer finds exact manual sections. The resolver then turns those passages into an ordered diagnostic answer instead of giving only generic suggestions.

3. Training and Testing Result

Metric	Result
Total extracted examples	730
Training examples	584
Testing examples	146

Metric	Result
Classifier test accuracy	61.64%
Trained model file	ai_output/manual_classifier_model.joblib
Manual index file	ai_output/manual_assistant_index.json

The current labels are automatically generated from the manuals, so the accuracy is a realistic prototype result. In a production version, the next step would be to add manually reviewed technician labels and real service-ticket examples.

4. Dashboard

The dashboard is a local web page where the user writes the equipment problem in normal text. When served with `python serve_dashboard.py`, it calls the Python resolver API, detects the issue area, searches the manuals, and shows detailed possible causes, diagnostic checks/fixes, safety notes, and source evidence.

Dashboard input	Dashboard output
The yacht air conditioning has low pressure and the fresh water flow alarm appears. What could be the reason and how can I solve it?	Detected area: cooling_water Detailed answer: The manuals point first to a circulation problem, not immediately to replacing parts. Possible reasons: no/low fresh-water circulation, trapped air, dirty filter/strainer, closed or unbalanced valve, pump/seawater-flow issue, water leak, or incorrect circuit pressure. Diagnostic checks: check flow-sensor status, verify pump operation, bleed the system, clean filter/strainer, inspect valves/manifold balance, check leaks and pressure, then check seawater-side flow.

For this example, the assistant found supporting evidence from the Frigomar CU50VFD/CU70VFD manual and the AH Series fan-coil manual, including pages related to fresh-water flow alarm, pump/flow checks, strainer/filter cleaning, manifold balancing, water-circuit bleeding, and water-pressure limits.

Important run note: For the improved detailed answers, the dashboard should be opened through the local server, for example `python serve_dashboard.py` then `http://localhost:8765/dashboard.html`. Opening `dashboard.html` directly as a file uses the weaker browser fallback.

5. Files Created

File	Purpose
pdf_ai_classifier.py	Main training/build script.
resolve_issue.py	Tests the troubleshooting assistant from the command line.
classify_text.py	Tests only the trained classifier.
serve_dashboard.py	Runs the local dashboard server and resolver API.
ai_output/dashboard.html	User-facing dashboard.
ai_output/manual_classifier_model.joblib	Trained sklearn model.

6. Next Steps

This is a working sample. To make it production-ready, I would add more manuals, expert-labeled service examples, real fault tickets, better semantic search with embeddings, and optionally a GPT/LLM layer to write more natural answers while keeping every answer grounded in retrieved manual evidence and PDF/page references.

Summary: I created a sample AI troubleshooting system from the supplied PDFs. It trains a classifier, builds a searchable manual index, and provides a dashboard that helps answer yacht equipment issues using the manuals.